

JSON Schema Validation in rsonpath - Research Project Report

Ewa Kaluś

1 Motivation

JSON Schema saw a sharp rise in popularity after OpenAI introduced *structured outputs* [6] — a feature that forces the model to generate responses according to the provided JSON Schema. In autonomous systems, this creates a constant stream of machine-generated JSON documents that have to be validated before being passed downstream.

However, existing JSON Schema validators require the input JSON to be fully parsed before validating it against a schema. When parsing large files, the resulting DOM (Document Object Model) can exceed available system memory. Streamed validation avoids this problem by ensuring that the memory usage is proportional to the document's depth, rather than its size. By avoiding the DOM construction and reusing the fast SIMD-based structural classifier from rsonpath [2], the validator can also achieve higher throughput than methods that have to fully parse the input beforehand.

2 Background

JSON is a widely adopted data format for exchanging data across modern systems, but it offers no built-in way to enforce the data structure. JSON Schema [9] addresses this by providing a declarative vocabulary designed to define and constrain the data.

A schema definition itself is a JSON document. The provided keywords can constrain data both at the atomic and structural levels. Validation keywords such as `minimum` or `minLength` restrict atomic values — in this case setting the minimum value of an integer or the minimal length of a string. Meanwhile, applicator keywords such as `properties` or `items` define the inner structure of objects and arrays.

As an example, let's consider the schema definition in Listing 1. It demonstrates a subset of keywords supported by the validator. This schema accepts either an object or a null value. If the input is an object, the `properties` keyword declares that it may contain a `name` field, which must be a string and a `year` field, which must be a number. The `minProperties` keyword forces the object to contain at least one of those fields, while `"additionalProperties": false` forbids the object from containing any other keys.

```
{
  "type": ["object", "null"],
  "properties": {
    "name": { "type": "string" },
    "year": { "type": "integer" }
  },
  "minProperties": 1,
  "additionalProperties": false
}
```

Listing 1: An example JSON Schema.

3 Functionality

3.1 Supported JSON Schema keywords

The validator supports a subset of JSON Schema keywords, focusing on the most commonly used ones. Recent analysis of schemas from the JSON Schema Store, a repository of schemas for popular JSON documents (e.g., configuration files), shows that just five of the keywords supported by our validator (`type`, `$ref`, `properties`, `additionalProperties`, `items`) account for 65% of all keyword usage. Excluding metadata keywords that do not affect validation, such as `title` or `description`, this share jumps to 76% [5].

The `type` keyword verifies that the values match their expected primitive types. In objects, `properties` and `additionalProperties` map object fields to specific schemas, while `minProperties` and `maxProperties` bound the total number of properties within an object. In arrays, the `items` keyword provides the schema describing the array elements, while `minItems` and `maxItems` bound the array's length. To handle schema reuse, the validator resolves local `$ref` references pointing to definitions within a `$defs` block.

3.2 Well-formedness check

To process the data as fast as possible, the validator operates under the assumption that the incoming JSON is already well-formed. Therefore, it only validates the data against the provided schema. However, it is possible to explicitly turn on well-formedness checking if needed. When enabled, it verifies whether the structural characters and atomic values form a valid JSON. It rejects invalid syntax such as mismatched brackets or trailing commas, which would be ignored with the default settings. It does not check the correctness of the atomic values themselves, providing a good balance between the extended validation and the overhead of it.

4 Architecture

The validator's architecture is similar to rsonpath: the schema is first compiled into an automaton, and then the engine executes the provided automaton over streamed input.

4.1 Automaton

The JSON Schema is first compiled into an automaton. A single schema definition can accept multiple types (e.g., `"type": ["object", "null"]`), while listing constraints for all of them. The specification describes how to handle such cases: if the type of the instance is not the same as the type targeted by the keyword, it is considered to conform to the assertion [9, Section 7.6.1].

Since almost all validation keywords apply to exactly one primitive type (with the exception for `type`, `const`, and `enum`), we group them within a definition into distinct, type-specific *constraint states*. These states also store type-specific applicator keywords, such as `properties` and `additionalProperties` for objects or `items` for arrays, which correspond to transitions to nested subschemas. By

grouping constraints this way, the engine only performs a single type check upfront, completely eliminating the need to repeatedly check if the value's type matches the target type of each individual keyword.

At the root of every subschema automaton is a *type dispatch state*. The engine uses this dispatch state as the entry point, checks the value's type, and transitions to the corresponding constraint state.

Figure 1 shows the automaton created for the example schema in Listing 1. The execution starts at the root type dispatch state. If the engine detects an object, it transitions to the C_{obj} constraint state. This state contains everything needed to validate the object: it has outgoing transitions to properties and stores the `minProperties` constraint, which the engine will check when exiting the object. For every nested property, the pattern repeats: we transition to a type dispatch state, check the primitive type and transition to a constraint node (C_{name} or C_{year}) if the check succeeds. However, if the initial value is a null, the root dispatch state transitions directly to C_{null} . Thanks to this, the engine can safely ignore any other keywords, such as `properties` or `minProperties`, listed at the same level in schema definition.

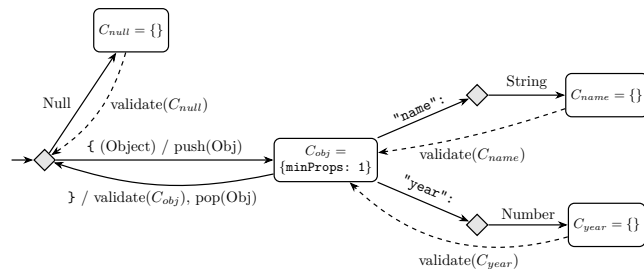


Figure 1: Automaton for the schema in Listing 1. Diamond nodes represent the type dispatch states, while rectangular nodes represent constraint states. Dashed lines represent ϵ -transitions.

4.2 Parser

The parser converts the provided JSON Schema into an automaton. It proceeds in two passes over the schema. In the first pass, it locates definitions inside the `$defs` block and reserves placeholder states for them. Then it parses these definitions, linking any references directly to the reserved nodes. This step resolves all recursive references. In the second pass, the parser traverses the remaining schema DOM recursively. It converts type keywords into dispatch states and groups constraints within a subschema into type-specific constraint states.

4.3 Engine

The engine executes the automaton over the provided input. It takes structural characters from the fast structural character iterator, reused directly from `rsonpath`. It performs a different action for each of the structural characters it encounters:

Opening. The engine saves the current state to the stack, resets the item counter to zero, and transitions to the subschema.

Closing. It evaluates all constraints that cannot be checked when the subtree opens (e.g., verifying if the element count satisfies `minItems`). After validation, it pops the stack to return to the parent schema.

Colon. Inside an object, the engine performs a backwards seek from the colon to find the closing quote of the property key. Then, it iterates over all keys provided in `properties` and compares the current key against every one of them, looking for a subschema to transition to. For the key comparison itself, the engine reuses the highly optimized `is_member_match` function directly from `rsonpath`. If no match is found, it checks the schema's policy for additional properties. Once the property's schema is resolved, the engine increments the property count.

Comma. In arrays, the engine increments the item count and transitions to the next element's schema. Commas are ignored in objects unless well-formedness check is active, as colons are fully sufficient for validating object properties.

The engine also performs type validation. Working under the assumption that the input JSON is valid, the primitive type of a value can be identified based on its first character. Every primitive type begins with a distinct character: `{` for objects, `[` for arrays, `"` for strings, `-` and `0-9` for numbers, `t` for true, `f` for false, `n` for null. By relying on this check, there is no need to fully parse atomic values, as only the first non-whitespace character after a structural character is needed.

Similarly to the engine in `rsonpath`, the validator engine also contains a stack, used to handle nested schema definitions. Whenever a new subtree opens, the engine pushes a frame containing the current context to the stack. When that subtree closes, the context is restored. Every frame stores the last automaton state, the type of the opened bracket and the current element count for that subtree.

4.4 Well-formedness

If turned on, the well-formedness checker works concurrently with the schema validation. It checks if the structural characters and atomic values form a structurally valid JSON. The checker uses a separate stack that stores only the opened bracket types. Keeping this stack independent from the main engine stack is done on purpose, as we do not want to rely on the assumption, that the main engine will push a frame on every subtree opening.

To avoid doubling work that was possibly already done by the main engine while checking schema compliance, the well-formedness checker uses some optimizations. To know if there is an atomic value between two structural characters, the checker keeps track of the position of the previous and current structural character, as well as last seen possible atomic value. When the main engine reads the initial character of a value to check its type, the checker updates its last seen atomic value to avoid doing the same seek.

5 Experiments

All experiments were done on a Dell Pro Max Tower T2 FCT2250 equipped with an Intel Ultra 9 285K processor and 32 GB of RAM. The benchmarks were ran using Criterion harness, which automatically handles execution warm-ups and statistical analysis. Every experiment records the end-to-end execution time, capturing file loading, schema compilation, and the validation itself. For *rschema*, the schema compilation of the datasets' schemas takes only a few microseconds, so the results can be treated as equal to working with a precompiled schema.

5.1 Datasets

The data used for benchmarks comes from the slightly modified DJV benchmark suite [4]. The datasets were created by concatenating individual JSON records into large arrays of varying lengths. In the plots, the number in parentheses denotes the total number of records in that specific test. The schemas were adapted by removing all keywords unsupported by *rschema*. Three datasets were chosen, each representing differently structured data:

- Corporations — The Corporations dataset consists of flat records with a maximum depth of 2.
- Chicago Crime — The Chicago Crime dataset has a more diverse structure, with a maximum depth of 4.
- Clang AST — The Clang AST dataset is deeply nested with a maximum depth of 280, and is generated directly from compiler output. This dataset specifically causes all of the DOM-based validators to fail entirely due to stack overflow.

Detailed characteristics of the datasets are shown in Table 1. By verbosity we mean the ratio of the size of the document to the number of nodes in the underlying tree.

Table 2 shows the characteristics of schemas describing the aforementioned datasets. We focus on the structural complexity of the schemas. We describe this complexity using three metrics: *objects* is the total number of distinct object definitions within the schema, *root properties* is the number of properties defined in the top-most object, while *average properties* refers to the mean number of properties per object definition.

Table 1: Datasets used in experiments.

Dataset	Depth	Records	Size (MB)	Verbosity
Corporations	2	10k	4.4	32.7
		100k	43.2	32.5
		200k	86.6	32.5
Crime	4	10k	6.5	21.9
		100k	65.6	21.9
		200k	131.2	21.9
Clang AST	280	10	44.0	15.5
		50	458.0	17.0
		100	731.6	16.3

Table 2: Schemas of datasets used in experiments.

Schema	Objects	Root Props	Avg Props
Corporations	1	36	36.0
Crime	3	24	10.3
Clang AST	4	5	4.0

5.2 Well-formedness checking overhead

To evaluate the computational overhead of well-formedness checking, we compare *rschema* on all datasets in three configurations. The first variant isolates well-formedness checking by validating the input against a generic true schema with well-formedness checking active, appearing in the legend as "WF only". The second runs normal schema validation with default settings, without any well-formedness checks. The third configuration, labeled "with WF", performs both normal schema validation and well-formedness checking. For each of these variants, we use both standard file reading and memory mapping (*mmap*) to compare how it affects the throughput.

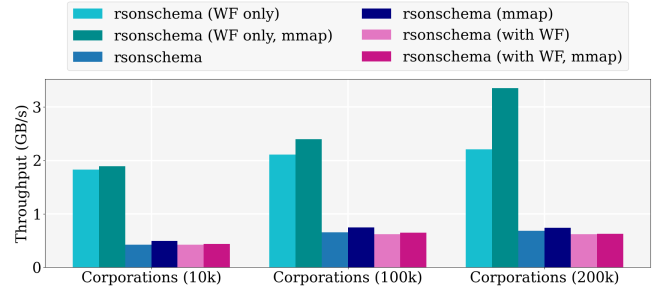


Figure 2: Overhead of well-formedness checking for *rschema* on the Corporations dataset.

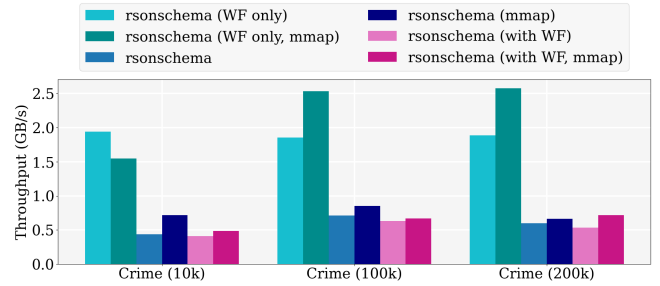


Figure 3: Overhead of well-formedness checking for *rschema* on the Chicago Crime dataset.

Running the well-formedness checker in isolation yields high throughput, peaking with almost 3.5 GB/s at the flattest Corporations dataset when using *mmap* (Figure 2). The throughput of well-formedness checking scales with the verbosity of the data, since the checker depends only on structural characters and atomics, while full schema validation must also match all the keys within

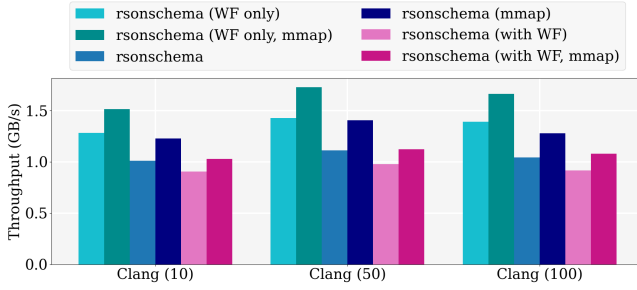


Figure 4: Overhead of well-formedness checking for rsonschema on the Clang AST dataset.

objects to their schemas. In all cases, the overhead of adding well-formedness validation to schema validation is quite low, showing the effectiveness of having the checker reuse the main engine’s operations whenever possible.

When using `mmap`, the well-formedness only throughput scales with the input size, providing additional speedup. The scaling is most likely caused by the fixed OS overhead of mapping memory pages. Because the WF only check executes so fast, its performance is visibly affected by this cost, which gets amortized over larger inputs. However, for the two smallest datasets, this speedup is negligible and even reversed. This is likely due to the same OS overhead, combined with the instability of measuring fast throughput on small files.

5.3 Validator comparison

We benchmarked `rsonschema` against four existing JSON Schema validators. Two of these are the most popular validators implemented in Rust - `jsonschema` [1] and `boon` [7]. We also included `Blaze` [8] - a highly optimized validator written in C++ and `DJA` [3], the only other known streaming JSON Schema validator. `DJA` was implemented in Rust, but is distributed only as a binary and no source code is available. Thus, the only way to fairly compare its performance with other validators is to measure time end-to-end, as there is no way to separate the file loading or schema compilation time.

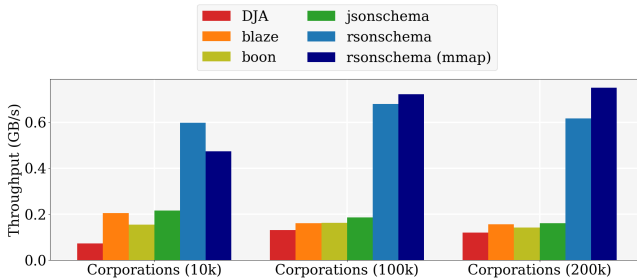


Figure 5: JSON validation throughput on the Corporations dataset

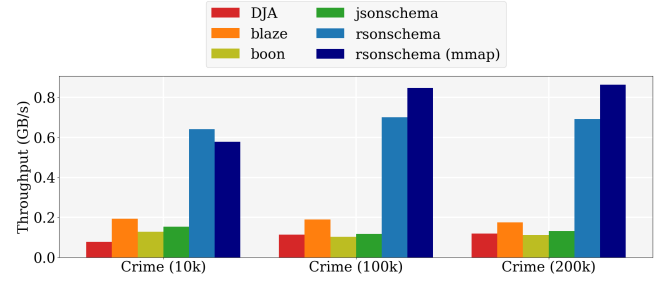


Figure 6: JSON validation throughput on the Chicago Crime dataset

For both the Corporations and Crime datasets (Figures 5 and 6), `rsonschema`, even without `mmap`, is consistently over 3× faster than other validators on the same dataset and input size.

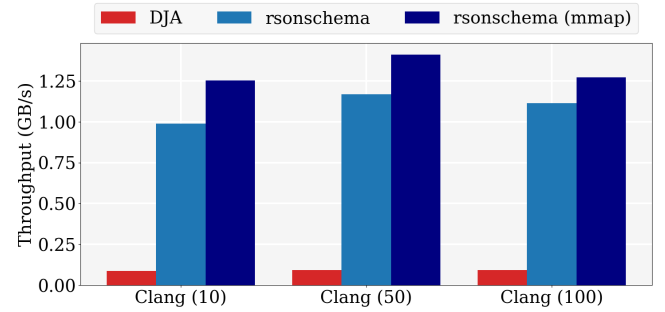


Figure 7: JSON validation throughput on the Clang AST dataset

In case of the Clang AST dataset (Figure 7), the performance difference is even more pronounced. Because of its extreme depth (up to 280), the only other validator able to process this data without crashing was the `DJA`.

These results show that `rsonschema`’s main bottleneck is property key matching. Throughputs across all datasets scale inversely to the number of root and average properties within the dataset’s schema, almost completely ignoring its verbosity, which was the main factor in the well-formedness only checking. This exposes an opportunity for further optimization of the validator engine’s performance, by replacing the linear scan through all property keys with a more efficient lookup.

Currently, for flat datasets with schemas where the majority of properties is defined in the root object, such as Corporations and Crime, the root properties metric is the best predictor of performance. In these datasets, most of the key comparisons are performed within the top-most object. Conversely, for deeply nested datasets with recursive schemas, such as Clang AST, the average properties metric is a better predictor, since the engine traverses multiple different object definitions, so the number of key comparisons distributes more evenly across all objects.

References

- [1] Dmitry Dygalo. 2026. *jsonschema: JSON Schema validation library for Rust*. <https://crates.io/crates/jsonschema>
- [2] Mateusz Gienieccko, Filip Murlak, and Charles Paperman. 2023. Supporting Descendants in SIMD-Accelerated JSONPath. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. ACM, Vancouver BC Canada, 338–361. doi:10.1145/3623278.3624754
- [3] Alexis Le Glaunec, Angela W. Li, and Konstantinos Mamouras. 2025. Streaming Validation of JSON Documents Against Schemas. *Proceedings of the VLDB Endowment* 19, 3 (Nov. 2025), 509–522. doi:10.14778/3778092.3778109
- [4] Alexis Le Glaunec, Angela W. Li, and Konstantinos Mamouras. 2025. *Streaming Validation of JSON Documents Against Schemas - Artifact*. doi:10.5281/zenodo.17665294
- [5] Simon Mikulcik. 2025. *86% of JSON Schemas Are Stuck in 2017 — Here's What That Tells Us About the Industry*. <https://medium.com/@smikulcik/86-of-json-schemas-are-stuck-in-2017-heres-what-that-tells-us-about-the-industry-58e1f74cddff>
- [6] OpenAI. 2024. *Introducing Structured Outputs in the API*. <https://openai.com/index/introducing-structured-outputs-in-the-api/>
- [7] Santhosh Kumar Tekuri. 2024. *boon*. <https://crates.io/crates/boon>
- [8] Juan Cruz Viotti and Michael J. Mior. 2025. Blaze: Compiling JSON Schema for 10x Faster Validation. *Proceedings of the VLDB Endowment* 19, 2 (Oct. 2025), 279–291. doi:10.14778/3773749.3773764
- [9] Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. 2020. *JSON Schema: A Media Type for Describing JSON Documents*. Technical Report. <https://json-schema.org/draft/2020-12/json-schema-core.html>